HetMap: Heterogeneous Graph Transformers for Code-to-Architecture Mapping

Rakhshanda Jabeen
Linnaeus University
Sweden
rakhshanda.jabeen@lnu.se

Morgan Ericsson
Linnaeus University
Sweden

Jonas Nordqvist
Linnaeus University
Sweden

Anna Wingkvist
Linnaeus University
Sweden

Abstract

Manual software architecture recovery is labor-intensive and time-consuming. Automating this process is essential to enable the practical application of static architecture compliance-checking techniques such as reflexion modeling. In this work, we explore a top-down architecture recovery approach that leverages partial architectural knowledge as supervision and learns to propagate it across the implemented system using graph neural networks.

We model each software system as a heterogeneous, multi-relational graph that captures a rich, implemented view of the code base. Nodes represent folders, files (classes), and finer-grained program elements in files such as methods, fields, and types. The edges encode both structural relations (e.g., folder hierarchy, containment) and code-level dependencies (e.g., calls, inheritance, field access). In this graph, we apply a Heterogeneous Graph Transformer (HGT) that learns meta-path-based attention across different relation types and node types. A small subset of entities, mapped to architectural modules from the intended architecture, is used as initial supervision, and the problem is formulated as a semi-supervised node classification task.

ACM Reference Format:
Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. HetMap: Heterogeneous Graph Transformers for Code-to-Architecture Mapping. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software systems evolve continuously, making it difficult to maintain accurate and up-to-date architectural documentation. As systems undergo refactoring, feature additions, and codebase changes, the implemented architecture may gradually diverge from the intended design. This architectural drift or erosion can negatively affect maintainability, understandability, reuse, and long-term evolution [2, 9].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Software Architecture Recovery (SAR) addresses this challenge by automatically inferring architectural structure from implementation artifacts. Recovered architectural views can support program understanding, re-engineering, and architecture conformance activities. In particular, Static Architecture Conformance Checking (SACC) techniques, such as reflexion modeling, compare implementation-level dependencies with architecture-level relations to identify convergent, divergent, and absent dependencies [18, 22]. However, these techniques require a mapping from source code entities to architectural modules. In practice, such mappings are often incomplete, unavailable, or not maintained as the system evolves [1]. Manually creating or updating them is labor-intensive, especially for large systems [13].

Code-to-architecture mapping is therefore closely related to architecture recovery. In both cases, the goal is to understand how implementation artifacts fit into a higher-level architectural decomposition. In this paper, we focus on the semi-supervised setting, where a small set of files is already mapped to architectural modules, while the remaining files are unmapped. The task is to use this limited architectural knowledge as supervision to infer the module assignments of the remaining implementation artifacts.

A central challenge is that architectural information is distributed across multiple implementation signals. Static dependencies capture how software entities interact, but dependency graphs alone can be noisy: shared utilities, framework code, and layered interactions may introduce many cross-module dependencies, so two files connected by a dependency do not necessarily belong to the same architectural module [29]. Dependency types may also carry different architectural meanings, as calls, inheritance, imports, and type-usage relations can reflect different forms of coupling [6, 20, 32]. At the same time, folder organization may reflect how developers structure the system into subsystems, while lexical information from file names and code identifiers can encode domain concepts and implementation roles [13, 35].

Automated code-to-architecture mapping has traditionally been studied as an orphan-adoption problem, in which an initially mapped set of source-code entities is used to assign the remaining unmapped entities to architectural modules [8, 33]. Recent machine-learning approaches, such as NBA, show that syntactic dependency and lexical information from source code can be effectively combined for semi-supervised mapping [26]. However, these approaches usually represent the available information as textual features or attraction terms. As a result, typed software relations, folder organization,

containment, and member-level information are not modeled as first-class graph elements that interact during learning.

To address this limitation, we propose *HetMap*, a Heterogeneous Graph Transformer-based approach for semi-supervised code-to-architecture mapping. We model each software system as a heterogeneous graph containing folder, file, and member nodes. Folder nodes capture project organization, file nodes represent the target software entities to be mapped, and member nodes summarize member-level information associated with each file. Edges represent structural relations, such as folder hierarchy and containment, as well as typed static dependencies extracted from the implementation.

We use the Heterogeneous Graph Transformer (HGT) because it conditions message passing on the source node type, relation type, and target node type [15]. This is important for software graphs, where folder containment, member containment, and static dependencies provide different forms of architectural evidence. Through type- and relation-aware attention, *HetMap* learns file representations from heterogeneous neighborhoods and predicts architectural module assignments for unmapped files using limited labeled data.

The goal of this paper is therefore twofold. First, we evaluate whether HGT-based heterogeneous graph learning can improve semi-supervised code-to-architecture mapping under limited supervision. Second, we analyze which software information sources and graph design choices contribute most to recovering ground-truth architectural modules. In particular, we study folder information, lexical identifier features, dependency structure, member-level information, positional encodings, and member representation granularity across systems implemented in Java, C, and C++.

The primary contributions of this work are as follows:

(1) **Semi-supervised architecture mapping formulation.** We formulate code-to-architecture mapping as a semi-supervised architecture recovery problem, where a small set of labeled files provides architectural anchors for inferring the module assignments of the remaining system.

(2) **Heterogeneous graph transformer framework.** We propose *HetMap*, an HGT-based framework that represents folder organization, files, summarized member-level information, containment relations, and typed static dependencies in a unified heterogeneous software graph.

(3) **Analysis of architectural information signals.** We conduct an ablation study to analyze how folder structure, lexical identifiers, static dependencies, member-level information, feature initialization, and member representation granularity affect mapping performance under limited supervision.

2 Background

Architecture conformance checking requires mapping source code entities to architectural modules. Such a mapping enables comparing the implemented system with the intended architecture and reasoning about architectural qualities such as maintainability, performance, and reuse. In techniques such as Reflexion modeling [22], implementation-level dependencies are compared with architecture-level dependencies to identify convergent, divergent, and absent relations. However, these techniques assume that source code entities have already been mapped to architectural modules. In practice, this mapping is often incomplete, unavailable, or not maintained as the system evolves.

We consider this mapping task in a semi-supervised setting, following prior work that begins with an initial set of mapped entities and assigns the remaining unmapped entities to architectural modules [8, 26, 33]. A small subset of entities is assumed to have known module labels, while the remaining entities have their labels inferred from information available in the implementation, such as structural dependencies and lexical or organizational signals.

Software artifacts are naturally relational and multi-typed. A system contains folders, source files or classes, and finer-grained program elements such as methods and fields. These artifacts are connected through different relations: folders may contain subfolders and files; files or classes may contain members; and software entities may interact through static code-level dependencies such as calls, inheritance, type usage, and field access. These information sources can provide architectural cues. Folder organization may reflect how developers structure the system and separate responsibilities, while code identifiers can encode domain concepts and implementation roles [13, 35]. Similarly, different dependency types may carry different architectural meanings; for example, function-call edges may indicate stronger architectural coupling than type-usage edges [6, 32].

To preserve these distinctions, we represent each software system as a heterogeneous software graph $G = (V, E, \mathcal{A}, \mathcal{R})$. Here, V denotes the set of software artifact nodes and E denotes the set of relations between them. Each node $v \in V$ is assigned a node type through the mapping function $\tau(v) : V \rightarrow \mathcal{A}$, where $\mathcal{A}$ may include artifact types such as folders, classes, methods, and fields. Similarly, each edge $e \in E$ is assigned a relation type through $\phi(e) : E \rightarrow \mathcal{R}$. These relations may capture folder hierarchy, the association between folders and classes, the association between classes and finer-grained program elements, and static code dependencies such as method calls, inheritance, and imports.

In this graph, the meaning of a relation is not characterized by the relation type alone. It also depends on the types of the source and target nodes that the edge connects. Therefore, for an edge $e_{s,t}$ from source node s to target node t, we describe the typed interaction as a meta-relation $\langle \tau(s), \phi(e_{s,t}), \tau(t) \rangle$. For example, a $\langle$folder, *contains*, file$\rangle$ relation captures project organization, whereas a $\langle$file, *Import*, file$\rangle$ relation captures code-level coupling. Thus, the same graph can preserve both structural organization and implementation dependencies without treating them as identical relations.

The Heterogeneous Graph Transformer (HGT) [15] is designed to exploit such meta-relations during message passing. It conditions attention and message transformation on the source node type, edge type, and target node type, making it suitable for code-to-architecture mapping, where different software relations provide different evidence for module assignment.

3 Methodology

In this section, we present *HetMap*, a heterogeneous graph-based approach for semi-supervised code-to-architecture mapping. We begin by formulating the mapping task, then construct the heterogeneous software graph from the project organization and static dependencies. We then describe the node feature initialization strategies, the HGT-based encoder, and the iterative self-training framework for inferring architectural module assignments.

3.1 Problem Formulation

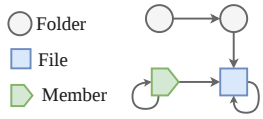
We formulate the code-to-architecture mapping as a semi-supervised node classification problem over a heterogeneous software graph $G = (V, E, \mathcal{A}, \mathcal{R})$. Let $V_f \subseteq V$ denote the set of target file nodes. A small subset $L \subset V_f$ is initially labeled with known architectural module assignments, while the remaining file nodes $U = V_f \setminus L$ are unlabeled. The objective is to learn an encoder f_θ that maps the heterogeneous graph to a d-dimensional latent embedding space:

$$Z = f_\theta(G), \qquad Z \in \mathbb{R}^{|V| \times d} \tag{1}$$

where each row of Z is the learned representation of a software entity. The embeddings of the target file nodes are then used to predict their architectural modules and infer the labels of the unlabeled file nodes in U.

3.2 Heterogeneous Graph Construction

We model the software system as a heterogeneous graph G with nodes representing software artifacts and edges corresponding to organizational, containment, and dependency relations. The graph is constructed from two main sources of information: the folder structure and the static dependencies extracted from the source code. The folder structure provides organizational context, while static dependencies describe how software entities interact. Since these relations capture different aspects of the system, we preserve their types instead of collapsing them into a single homogeneous relation. Figure 1 illustrates the general schema of the heterogeneous software graph and the meta-relations used in our model.



(a) Software graph schema

Folder	*parent_of*	Folder
Folder	*contains*	File
File	*depends_on*	File
Member	*belong_to*	File
Member	*depends_on*	Memeber

(b) The meta relations

Figure 1: Schema and meta-relations of the heterogeneous software graph. The figure shows the most expressive representation, with folder, file, and member nodes. Simpler variants remove selected node types or aggregate member-level dependencies at the file level.

3.2.1 Node Types. We consider three node types: folder, file, and member. Folder nodes represent directory levels in the project structure. File nodes represent source files or classes and are the target nodes for architecture mapping. Member nodes represent member-level program information associated with a file, such as methods, fields, constructors, and variables.

To keep the graph schema scalable, we use a single *Member* node type rather than modeling methods, fields, constructors, and variables as separate node types. In the main graph configuration, member-level information is represented by associating each file with one member node. This design preserves member-level lexical and dependency information while avoiding the large and sparse graphs that arise when every fine-grained program element is represented as a separate node.

3.2.2 Node Types. We consider three node types: folder, file, and member. Folder nodes represent directory levels in the project structure. File nodes represent source files or classes and are the target nodes for architecture mapping. Member nodes represent member-level program information associated with a file, such as methods, fields, constructors, and variables.

To keep the graph schema scalable, we use a single *Member* node type rather than modeling methods, fields, constructors, and variables as separate node types. In the main graph configuration, the member-level information for each file is summarized into a single member node connected to that file. This design preserves member-level lexical and dependency information while avoiding the large and sparse graphs that arise when every fine-grained program element is represented as a separate node.

3.2.3 Relation Types. The graph contains structural and dependency relations. Structural relations capture organization and containment. Logical components in software systems are often implemented through namespaces or directory structures that contain source code providing a particular functionality. Such structures reflect how developers organize files and can help an architect reason about the system's logical structure [13, 27]. To preserve this organizational information, each directory level is represented as a separate folder node. Folder nodes are connected through *parent_of* edges, forming hierarchical chains such as `app` → `app/ui` → `app/ui/views`. File nodes are linked to their corresponding folder nodes through *contains* edges. When member nodes are included, each member node is connected to its enclosing file through a *belongs_to* edge.

Static dependencies provide direct evidence of how software entities interact and are therefore a central source of information for architectural mapping and recovery tasks [6, 35]. We use the Depends tool [23] to extract these dependencies. Depends analyzes the syntactic structure of Java and C/C++ projects and identifies software entities together with several categories of static relations. Since different dependency types can carry distinct architectural meanings [32], we preserve them as separate relational edges. Table 1 summarizes the structural relations introduced during graph construction and the static dependency types observed in the analyzed Java and C/C++ systems.

3.2.4 Graph Variants. To analyze how different structural information sources affect mapping performance, we construct four variants of the heterogeneous software graph. The variants differ in whether they include folder structure, a separate member-level representation, or only file-level dependencies. Table 2 summarizes the variants.

The FFM variant corresponds to the complete graph used in our design. It includes folder, file, and member nodes, together with

Table 1: Structural and static relation types used in the software graph. Checkmarks indicate whether a relation type is observed in the analyzed Java and C/C++ systems.

Relation type	Description	Java	C/C++
parent_of	Folder contains a subfolder in the directory.	✓	✓
contains	Folder contains a source file.	✓	✓
belongs_to	Member-level program element belongs to a file.	✓	✓
`Annotation`	Annotation relation to an annotated entity.	✓	–
`Call`	Function or method invocation.	✓	✓
`Cast`	Conversion of an entity to another type.	✓	✓
`Contain`	Definition or containment of variables/fields.	✓	✓
`Create`	Creation of an instance or program entity.	✓	✓
`Extend`	Inheritance or parent-child type relation.	✓	✓
`Implement`	A class implements an interface or abstraction.	✓	✓
`Import`	A file imports other classes, methods, or variables.	✓	✓
`Parameter`	Use of a type as a method or function parameter.	✓	✓
`Return`	Use of a type as a return type.	✓	✓
`Throw`	A type is used as an exception.	✓	–
`Use`	Use or assignment of variables.	✓	✓

Table 2: Graph variants used to study the effect of graph granularity and information sources.

Variant	Node types	Purpose
FFM	Folder, File, Member	Complete heterogeneous structure
FF	Folder, File	Folder and file-level structure
FM	File, Member	Member-level structure without folders
File-only	File	File-level dependency baseline

structural relations and typed static dependency relations. Each member node summarizes the member-level information of one file and is connected to its corresponding file through a *belongs_to* edge. Static dependency edges are modeled separately from this containment relation. Thus, file–file, member–member, file–member, and member–file dependencies are preserved according to their resolved endpoints.

The FM variant removes folder nodes and folder relations, but keeps file and member nodes. Thus, it preserves member-level information and static dependencies involving member nodes without using the project folder hierarchy. The FF and File-only variants remove member nodes. In these variants, member-level dependencies are projected to the file level by connecting the files that contain the original dependency endpoints. The FF variant keeps folder nodes and folder-to-file relations, whereas the File-only variant keeps only file nodes and file-level dependency edges.

3.3 Node Feature Initialization

We initialize node features based on the information source used in each experimental setting. Folder nodes are initialized from file path tokens derived from directory segments, while file and member nodes are initialized using lexical features, structural positional features, or their combination.

3.3.1 Folder Node Features. Folder nodes are encoded using a binary bag-of-words representation over folder tokens. In the semantic setting, folder tokens are derived from directory segments after removing common project-root tokens. Each folder node is represented by the tokens along its path in the directory hierarchy. For example, a folder corresponding to `app/ui/views` carries the tokens `app`, `ui`, and `views`. This allows deeper folder nodes to retain information about their hierarchical context.

As a control setting, we also use randomized folder tokens while preserving the original folder hierarchy and file connections. In this setting, each folder is assigned a random token, so the model can still use the directory structure but not the semantic meaning of folder names such as `gui`, `model`, or `util`.

3.3.2 Word2Vec Lexical Features . Source code identifiers often encode domain concepts and implementation roles, making them useful lexical cues for mapping software entities to architectural modules and for related architecture recovery tasks [35]. To encode this information, we extract tokens from file names and from member-level program elements such as methods, fields, constructors, and variables. Identifiers are normalized by splitting camelCase, PascalCase, and snake_case forms, lowercasing the resulting tokens, and removing stopwords and non-alphabetic tokens. The remaining tokens are embedded into 128-dimensional vectors using the skip-gram variant of Word2Vec [21], trained for 20 epochs with a context window of five. A node representation is obtained by averaging the token embeddings associated with that node.

The use of lexical features depends on the graph variant. In the FFM and FM graphs, file nodes are initialized from file-name tokens, while member nodes are initialized from tokens extracted from the program elements associated with each file. Thus, member-level lexical information reaches file nodes through member-to-file relations and member-level dependency edges.

In the FF and File-only graphs, member nodes are not included, so member-level tokens are aggregated into the corresponding file node features. This allows file-level variants to retain lexical information from methods, fields, constructors, and variables without explicitly modeling member nodes.

3.3.3 Laplacian Positional Encoding. As a structural alternative to lexical initialization, we use Laplacian positional encoding (LPE) [10]. It encodes a node's position in the dependency graph. Thus, LPE allows us to initialize nodes solely from the graph structure and to examine whether the dependency topology carries useful architectural information for code-to-architecture mapping.

Intuitively, LPE assigns each node a set of structural coordinates. Nodes that occupy similar positions in the graph tend to receive similar coordinates, even if their names or identifiers are different. For example, in a simple chain of dependencies, the first positional coordinate may separate nodes near one end of the chain from those near the other, while another coordinate may distinguish central nodes from boundary nodes. In this way, LPE provides the model with information about global and local graph positions.

For a graph with adjacency matrix A and degree matrix D, we first symmetrize the dependency graph and compute the normalized graph Laplacian $L = I - D^{-1/2}AD^{-1/2}$. The eigenvectors of this matrix serve as the node positions. Eigenvectors associated with smaller non-zero eigenvalues capture smoother structural patterns in the graph, with connected nodes tending to have similar values. We use the first k non-trivial eigenvectors as node features, with $k = 30$ in our experiments.

Since the graph variants differ in node types and dependency granularity, LPE is computed on the dependency structure relevant to each variant. In FFM and FM, file-node encodings are computed from file-to-file dependencies, while member-node encodings are computed from member-level dependencies. In FF and File-only, member nodes are absent; therefore, all dependencies are projected to their enclosing files, and LPE is computed on the resulting file-level dependency graph.

3.3.4 Laplacian Positional Encoding . As a structural alternative to lexical initialization, we use Laplacian positional encoding (LPE) [10]. It encodes a node's position in the dependency graph. Thus, positional encoding allows us to initialize nodes solely from the graph structure and to examine whether the dependency topology carries useful architectural information for code-to-architecture mapping.

Intuitively, LPE assigns each node a set of structural coordinates based on the graph's structure. Nodes that occupy similar positions in the graph tend to receive similar coordinates, even if their names or identifiers are different. For example, in a simple chain of dependencies, one coordinate may separate nodes near one end of the chain from those near the other, while another coordinate may distinguish middle nodes from end nodes. Formally, for a graph with adjacency matrix A and degree matrix D, we first symmetrize the dependency graph and compute the normalized graph Laplacian $L = I - D^{-1/2}AD^{-1/2}$. The eigenvectors of this matrix are used as node features. Eigenvectors associated with smaller non-zero eigenvalues capture smoother structural patterns, where connected nodes tend to have similar values. We use the first k non-trivial eigenvectors as node features, with $k = 30$ in our experiments.

Since the graph variants differ in node types and dependency granularity, LPE is computed on the dependency structure relevant to each variant. In FFM and FM, file-node encodings are computed from file-to-file dependencies, while member-node encodings are computed from member-level dependencies. In FF and File-only, member nodes are absent; therefore, all dependencies are projected to their enclosing files, and LPE is computed on the resulting file-level dependency graph.

3.4 *HetMap* Architecture

The *HetMap* model consists of three components: a type-specific input projection, an HGT layer, and a file-level classification head. The model operates on the heterogeneous software graph defined in Section 3.2 and uses the Heterogeneous Graph Transformer (HGT) [15] as its encoder.

In our setting, architectural module assignments are predicted for file nodes. Folder and member nodes are included as contextual nodes: folder nodes provide organizational information through the directory structure, while member nodes provide member-level code information through their association with files and member-level dependencies. Through HGT-based message passing, these contextual signals are propagated through the graph and used to enrich the final file-node representations.

3.4.1 ***Type-Specific Input Projection.*** Folder, file, and member nodes have different feature spaces depending on the graph variant and feature initialization strategy. Therefore, before message passing, each node type is mapped into a shared hidden dimension using a dedicated type-specific input transformation as follows:

$$H_\tau^{(0)} = \mathrm{ReLU}(X_\tau W_\tau + b_\tau) \tag{2}$$

where X_τ denotes the input feature matrix of node type τ, and W_τ and b_τ are type-specific learnable parameters. The ReLU activation introduces non-linearity after the type-specific linear layer. This transformation allows all node types to be processed in a common latent space while preserving type-specific feature mappings.

3.4.2 ***Heterogeneous Graph Transformer Layer.*** After type-specific input projection, we apply a Heterogeneous Graph Transformer (HGT) layer [15], which updates node representations through node-type- and relation-aware message passing over heterogeneous neighborhoods. For each target node, HGT computes attention-weighted messages from neighboring source nodes, where both the attention scores and the message transformations depend on the source node type, edge type, and target node type. Following the self-attention mechanism [34], the attention scores are computed using query and key projections, which are adapted in HGT to account for node and relation types.

Formally, let $h_t^{(0)}$ denote the projected representation of target node t, and let $s \in \mathcal{N}(t)$ be a source node connected to t by edge $e = (s, t)$. For one attention head, HGT first computes an attention weight $\alpha(s, e, t)$ that determines how strongly node s contributes to updating node t. The normalized attention weight is computed as:

$$\alpha(s,e,t) = \underset{s\in\mathcal{N}(t)}{\mathrm{softmax}}\left(\frac{K_{\tau(s)}(h_s^{(0)})W_{\phi(e)}^{ATT}\left(Q_{\tau(t)}(h_t^{(0)})\right)^\top}{\sqrt{d}} \cdot \mu_{\langle\tau(s),\phi(e),\tau(t)\rangle}\right) \tag{3}$$

Here, $Q_{\tau(t)}$ and $K_{\tau(s)}$ are type-specific query and key transformations, $W_{\phi(e)}^{ATT}$ is an edge-type-specific attention matrix, and $\mu_{\langle\tau(s),\phi(e),\tau(t)\rangle}$ is a learnable prior for the corresponding meta-relation. The value d denotes the dimensionality of one attention head. In multi-head attention, the hidden representation is split evenly across attention heads; therefore, the hidden dimension is chosen to be divisible by the number of heads.

In parallel, HGT computes a transformed message $M(s, e, t)$, which represents the information sent from source node s to target node t. This message is obtained using a source-type-specific transformation followed by an edge-type-specific transformation as follows:

$$M(s,e,t) = M_{\tau(s)}(h_s^{(0)})\, W_{\phi(e)}^{MSG}. \tag{4}$$

The HGT layer then aggregates the attention-weighted messages from the heterogeneous neighborhood of t as follows:

$$\widetilde{h}_t = \sum_{s\in\mathcal{N}(t)} \alpha(s,e,t)\cdot M(s,e,t). \tag{5}$$

This formulation allows messages passed through folder, member, and static dependency relations to be weighted and transformed differently before contributing to the updated node representation.

The HGT layer output is then combined with the projected input representation via a residual connection, followed by layer normalization. The residual connection adds the projected input representation back to the message-passing output, so the updated node representation preserves information from the previous representation while incorporating information from its heterogeneous

neighborhood [14]. The final representation of node t after one HGT layer is computed as:

$$h_t^{(1)} = \text{LayerNorm}\left(\widetilde{h}_t + h_t^{(0)}\right). \tag{6}$$

3.4.3 ***Module Prediction Layer.*** Architectural module assignments are predicted for file nodes. Folder and member nodes are not directly classified; instead, they provide contextual information during HGT-based message passing. We denote the final node representations produced by the HGT encoder as $Z = H^{(1)}$, and the file-node representations as Z_f. These file-node representations are passed to a linear prediction layer:

$$\hat{Y}_f = Z_f W_c + b_c, \tag{7}$$

where W_c and b_c are learnable prediction-layer parameters. The output $\hat{Y}_f$ contains the module logits for file nodes. During supervised training, the model is optimized on the labeled training file nodes to minimize the cross-entropy loss.

3.5 Iterative Learning Framework

Starting from a small initially labeled set of file nodes, *HetMap* first trains the model in a supervised warm-up phase. The original labeled nodes are denoted by L, while the remaining file nodes form the unlabeled set U. After warm-up, the model predicts module probabilities for file nodes, and the confidence of a prediction is defined as the maximum softmax probability over the module classes.

In each self-training round, unlabeled file nodes whose confidence exceeds a threshold δ are assigned pseudo-labels and added to the pseudo-labeled set L_{pseudo}. The model is then retrained using both the original labeled nodes and the accepted pseudo-labeled nodes. Existing pseudo-labels are also revisited: if the model predicts a different label for a pseudo-labeled node with confidence above δ, its pseudo-label is updated. The process stops when no new confident pseudo-labels are found, fewer than five nodes were added in the previous self-training round, or the maximum number of self-training rounds is reached. Algorithm 1 summarizes the procedure.

Algorithm 1: Iterative Framework

Input: Software graph $G = (V, E, \mathcal{A}, \mathcal{R}, X)$; target file nodes V_f;
original labeled set $L \subseteq V_f$; unlabeled set $U = V_f \setminus L$;
warm-up epochs ϵ_{base}; self-training epochs ϵ_{self};
self-training rounds R; confidence threshold δ
Output: Predicted module assignments for file nodes in V_f

$L_{\text{pseudo}} \leftarrow \emptyset$;
$\mathcal{Y} \leftarrow L$;
$prev_added \leftarrow \infty$;
Initialize model parameters θ;
Phase 1: Supervised warm-up
for $e = 1$ **to** ϵ_{base} **do**
  $\hat{Y}_f \leftarrow f_\theta(G)$;
  Update θ using cross-entropy loss on L;
Phase 2: Iterative self-training
for $r = 1$ **to** R **do**
  if $prev_added < 5$ **then**
    break;
  $\hat{Y}_f \leftarrow f_\theta(G)$;
  Predict $\hat{y}_v$ and $\text{conf}(v)$ for each $v \in V_f$;
  $\mathcal{Y}(v) \leftarrow \hat{y}_v$ for all $v \in L_{\text{pseudo}}$ such that $\text{conf}(v) > \delta$ and $\hat{y}_v \neq \mathcal{Y}(v)$;
  $C \leftarrow \{v \in U \mid \text{conf}(v) > \delta\}$;
  if $C = \emptyset$ **then**
    break;
  Assign pseudo-labels $\hat{y}_v$ to nodes in M;
  $L_{\text{pseudo}} \leftarrow L_{\text{pseudo}} \cup M$;
  $U \leftarrow U \setminus M$;
  $prev_added \leftarrow |M|$;
  for $e = 1$ **to** ϵ_{self} **do**
    $\hat{Y}_f \leftarrow f_\theta(G)$;
    Update θ using cross-entropy loss on $L \cup L_{\text{pseudo}}$;
return *Predicted module assignments for file nodes in* V_f;

4 Experiments

We evaluate HetMap on a collection of open-source software systems with known ground-truth code-to-architecture mappings. The task is formulated as semi-supervised code-to-architecture mapping and is closely related to top-down architecture recovery, as it infers module assignments from limited architectural knowledge. Beyond evaluating mapping accuracy, we also study implementation signals commonly used in architecture recovery and code-to-architecture mapping, including folder organization, static dependencies, code identifiers, positional encodings, and member-level information.

The experimental study is structured to address the following research questions:

(1) **RQ1.** How effective is HetMap for semi-supervised code-to-architecture mapping under limited supervision?
(2) **RQ2.** How do implementation signals and heterogeneous graph representation choices affect mapping performance?
(3) **RQ3.** What is the computational cost of HetMap across systems of different sizes?

4.1 Software Systems

We evaluate *HetMap* on eleven open-source systems with available ground-truth (GT) code-to-architecture mappings. The systems have been used in prior work on software architecture recovery and C2A mapping [5, 26, 35]. They cover different programming languages, domains, sizes, and architectural granularities. Using published GT mappings is important because producing reliable C2A mappings requires substantial manual effort, architectural knowledge, and often expert validation [13]. For example, the Chromium mapping is particularly notable because it required long-term collaboration with developers over roughly two years [20]. These established mappings also support comparability with previous studies.

Table 3 summarizes the main system and graph properties used in our experiments. We also report file-level dependency homophily $\mathcal{H}$, which measures the fraction of static dependencies whose source and target files belong to the same ground-truth module:

$$\mathcal{H} = \frac{|\{(i,j) \in FD : y_i = y_j\}|}{|\text{Dependecies}|}, \quad 0 \leq \mathcal{H} \leq 1 \tag{8}$$

where (i, j) is a directed dependency and y_i and y_j denote the ground-truth modules of files i and j.

The Java systems are taken from the benchmark used by Olsson et al. [26]. JabRef and TeamMates (T.Mates) were studied at the Software Architecture Erosion and Architectural Consistency Workshop (SAEroCon) in 2016 and 2017, respectively, where system

Table 3: Properties and versions of software systems. *M* and Deps. denotes the architectural modules and total code-level static dependencies.

System	Version	Lang.	LoC	M	Folders	Files	Members	Deps.	$\mathcal{H}$
Ant	r584500	Java	36k	15	132	1 079	28.2k	44.7k	0.40
A.UML	r13713	Java	62k	14	57	766	20.1k	31.9k	0.63
C.Img	v1.0a2	Java	18k	21	84	503	9.7k	18.3k	0.69
JabRef	v3.7	Java	59k	6	122	1 180	17.7k	43.0k	0.61
Lucene	r1075001	Java	36k	7	330	1 500	37.8k	111.8k	0.69
SH-3D	r002382	Java	35k	9	11	191	10.9k	17.9k	0.44
T.Mates	v5.11	Java	55k	15	40	778	22.4k	67.0k	0.28
Bash	4.2	C	115k	14	12	292	3.7	1.8k	0.39
HDF	0e196f	C	153k	9	26	153	1.9k	3.7k	0.38
Chrome	svn-171054	C++	11M	69	999	18 343	513.3k	875.4k	0.73
HDC	46ff87	C++	26k	11	87	207	3.4k	3.2k	0.36

experts provided the architectures and mappings[1]. Ant, ArgoUML (A.UML), and Lucene were studied by Brunet et al. [5] and Lenhard et al. [19]. The Commons Imaging (C.Img) mapping was provided by a project contributor as a test case for the s4rdm3x tool [25]. The labels for these Java systems are distributed through the s4rdm3x GitHub repository[2].

For Bash, we use the C2A mapping curated by Garcia et al. [13]. This mapping was produced using a certification process in which researchers first recovered an initial architectural decomposition from documentation and source code, and then refined it through validation with system experts. The same line of work also provides the GT mapping for Chromium, which required long-term collaboration with developers over roughly two years [20].

The GT C2A mappings for HDF and HDC were developed in collaboration with industry partners and are included in the SARIF benchmark studied by Zhang et al. [35]. The mappings are distributed through the SARIF replication package on GitHub[3].

4.2 Baseline Methods

Naive Bayes Attract (NBA). We compare *HetMap* against the NBA approach proposed by Olsson et al. [26]. This approach formulates software entity mapping as a semi-supervised text classification problem, assigning each source code entity to an architectural module based on textual and dependency-derived information.

NBA combines source-code vocabulary, including package names, file names, method names, code identifiers, and string literals, with structural dependency information encoded via Concrete Dependency Abstraction (CDA). In CDA, dependencies between entities are transformed into textual terms that reflect relations between architectural modules. For example, if entity e_1 inherits from entity e_2 and the two entities belong to modules A and B, respectively, then a term such as *A-Inherits-B* is generated and added to the documents of both A and B. This allows the NBA to leverage both semantic and structural signals.

The resulting textual representation is converted into a bag-of-words representation and used to train a multinomial Naive Bayes classifier. NBA operates in an iterative, semi-supervised manner: starting from an initial set of mapped entities, the classifier predicts labels for unmapped entities, and high-confidence predictions are progressively incorporated into the labeled set.

NB-LocLex. To better isolate the effect of feature design from graph-based message passing, we introduce an additional non-graph baseline based on Naive Bayes. Unlike the NBA, this approach does not use dependency-derived CDA terms. Instead, it relies only on location and lexical signals. Location information is encoded as a binary bag-of-words representation derived from directory structure, while code identifiers and file names are represented using the same Word2Vec-based features described in Section 3.3. These representations are combined into a single feature vector for each entity and used to train a Naive Bayes classifier in the same iterative semi-supervised setting as NBA. This baseline allows us to assess how far improved textual and location-aware representations can go without introducing explicit graph modeling.

4.3 Experimental Configuration

We evaluate all methods in a low-label semi-supervised setting. For each run, the initial labeled set is constructed by randomly selecting 5% of the file entities. To avoid very small or disproportionately large training sets, we enforce a minimum of 30 labeled files and a maximum of 200 labeled files. If a ground-truth module is not present in the sampled set, we add a file from that module to ensure that all modules are initially represented.

All methods are evaluated over 100 independent runs using the same initial-set construction strategy described above. For *HetMap*, we use the iterative learning procedure described in Section 3.5. For the Naive Bayes baselines, we use the confidence threshold $\delta = 0.90$ following Olsson et al. [26]. The process continues until no additional unmapped files exceed the threshold.

For *HetMap*, we evaluate different graph structures and node feature initializations, as described in Sections 3.2 and 3.3. All *HetMap* configurations use the same HGT architecture: one HGT layer with 256 hidden dimensions and 8 attention heads. Training is performed with the Adam optimizer using a learning rate of 0.001, with 100 supervised warm-up epochs followed by 30 epochs per self-training round. Table 4 summarizes the graph-dependent input and self-training settings used in the reported *HetMap* configurations.

Table 4: Graph-dependent input and self-training settings for the reported *HetMap* configurations. The LPE dimension applies only when LPE features are used.

Graph Variant	LPE dim.	Self-training rounds	Threshold (δ)
FFM	16	5	0.95
FF	32	5	0.95
FM	32	5	0.90
File-only	32	3	0.90
Graph+Loc	32	2	0.90

4.4 Performance Evaluation

The subject systems exhibit imbalanced module distributions, with some architectural modules containing many files while others contain only a few. Therefore, accuracy alone can be misleading

[1] https://github.com/sebastianherold/SAEroConRepo
[2] https://github.com/h0bb3/s4rdm3x
[3] https://github.com/anonymous2f4a9d/SARIF_FSE23

because it may overemphasize performance on the majority of modules. We evaluate mapping performance using precision, recall, and F1 score with both macro- and micro-averaging.

Macro-averaged metrics are computed independently for each module and then averaged, giving equal importance to all modules regardless of their size. This makes macro-F1 useful for assessing how well a method handles smaller or underrepresented modules. Micro-averaged metrics aggregate predictions over all files before computing the score, giving a measure of overall mapping performance across the system. Reporting both macro- and micro-F1 allows us to assess both class-balanced and overall performance.

Since the initial labeled set is randomly sampled and neural training can introduce stochastic variation, each experimental setting is repeated multiple times. We report the median performance across runs, as it is less sensitive to outliers than the mean. When comparing methods or configurations, we report the median difference in F1 scores to quantify the practical performance gap.

To assess whether observed differences are systematic rather than due to run-to-run variation, we perform non-parametric statistical testing on the repeated-run results. When two methods are evaluated using the same initial labeled splits, we use the paired Wilcoxon signed-rank test; otherwise, we use the Wilcoxon rank-sum test. Statistical tests are applied to both macro-F1 and micro-F1 scores. The detailed comparisons and their interpretation are reported in the Results section.

5 Results

This section presents the experimental results for semi-supervised code-to-architecture mapping across the subject systems. We evaluate *HetMap* under limited supervision and compare it with Naive Bayes baselines. The results are organized around three questions: overall mapping effectiveness, the effect of implementation signals and heterogeneous graph representation choices, and computational cost.

We report macro- and micro-averaged F1 scores. Macro-F1 gives equal importance to all architectural modules and is therefore useful for assessing performance on smaller or underrepresented modules. Micro-F1 aggregates predictions over all files and reflects overall mapping performance.

5.1 Overall Mapping Performance (RQ1)

To answer RQ1, we evaluate whether *HetMap* can effectively map source code entities to architectural modules when only a small portion of entities is initially labeled. We compare the main full-graph folder–file–member (FFM) *HetMap* configurations, $W2V_{FFM}$ and LPE_{FFM} initialized with W2V and LPE node features, respectively, with two Naive Bayes baselines (NBA) under the same 5% low-label setting. For each dataset and method, including the Naive Bayes baselines (NBA_{CDA} and NB_{LocLex}), we run 100 experiments with different initial labeled sets; for *HetMap*, we also vary the random seed to capture stochastic effects from training and model initialization.

Table 5 reports the median macro-F1 and micro-F1 scores across these runs. Overall, both full-graph *HetMap* variants achieve strong performance across the studied systems. The results are particularly strong and stable for the Java systems, where both macro-F1 and micro-F1 are often close to or equal to 100%. This shows that *HetMap* can recover architectural mappings effectively even when only 5% of the entities are initially labeled.

Overall, the results show that the full heterogeneous graph representation achieves strong mapping performance across the systems studied. Both $W2V_{FFM}$ and LPE_{FFM} achieve high macro-F1 and micro-F1 scores on most datasets, with particularly strong results on Java Systems. In several of these systems, the median micro-F1 is close to 100%, indicating that most entities are assigned to the correct architectural module even with an initial supervision rate of 5%.

The improvement is especially visible when comparing *HetMap* with the CDA-based Naive Bayes baseline. NBA_{CDA} performs reasonably on some systems, but its macro-F1 scores remain low for several datasets, indicating difficulty in consistently recovering minority or less represented modules. This behavior is expected in low-label settings because module-level textual evidence and dependency abstractions may be sparse when only a few mapped entities are available. In contrast, *HetMap* uses the heterogeneous software graph to propagate information through typed folder–file–member relations. This allows file representations to be updated not only from their own initial features, but also from folder context, member-level identifiers, and typed dependency neighborhoods.

Compared with NB_{LocLex}, the full-graph *HetMap* variants also show a clear performance gain. NB_{LocLex} is a stronger baseline than NBA_{CDA} because it benefits from lexical and location-based cues, and it performs particularly well on systems where the folder structure closely reflects the architectural decomposition. However, *HetMap* generally improves both macro-F1 and micro-F1, suggesting that typed message passing provides additional value beyond a flattened feature representation. This is important because the full-graph model does not simply concatenate all information into file-level vectors. Instead, it preserves folder, file, and member entities as separate node types and lets HGT learn how information should flow across the corresponding meta-relations.

The macro-F1 results are particularly important because the subject systems contain imbalanced architectural modules. A high micro-F1 score can be dominated by large modules, whereas macro-F1 gives equal importance to each module. The strong macro-F1 values obtained by $W2V_{FFM}$ and LPE_{FFM} therefore indicate that *HetMap* is not only assigning many entities correctly overall, but is also able to recover mappings for smaller modules more effectively than the baselines.

For the C and C++ systems, *HetMap* also achieves strong overall results. HDF performs close to the Java systems, with high macro-F1 and micro-F1 scores. Bash and HDC achieve lower macro-F1 scores, mainly because their modules are very small, with only 1 or 2 entities, making some classes difficult to recover under limited supervision. However, their micro-F1 scores remain close to 90%, showing that most entities are still mapped correctly. Chrome is a special case: it contains more than 18K entities, yet *HetMap* starts from only about 200 initially labeled entities and still achieves strong results, indicating that the full FFM graph can support effective mapping even at a much larger scale.

Figure 2 summarizes the run-to-run distributions of macro-F1 and micro-F1 scores across the 100 experiments. Both full-graph *HetMap* variants generally outperform NB-LocLex, with the clearest and most stable gains observed for the Java systems. For C and

Table 5: Median macro-F1 (MA) and micro-F1 (MI) scores (%) across 100 runs. The upper block reports the Naive Bayes baselines, and the lower block reports the full-graph FFM *HetMap* variants initialized with W2V and LPE node features.

Method	Ant		A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
NBA_{CDA}	16	44	31	62	20	53	40	81	53	75	48	73	23	60	21	44	45	55	–	–	45	72
NB_{LocLex}	54	90	63	88	56	75	62	95	87	98	75	92	80	93	54	86	80	89	46	85	48	82
$W2V_{FFM}$	99	98	93	98	99	98	86	97	99	100	100	100	100	100	68	88	95	95	84	93	55	89
LPE_{FFM}	98	98	91	97	88	95	81	97	100	100	100	100	100	100	67	83	92	94	78	90	54	88

C++ systems, the distributions show more variation, especially in macro-F1. This variation should be interpreted with care because these language groups contain fewer systems, so difficult cases such as Bash and HDC have a stronger effect on the grouped distribution. Their high micro-F1 scores nevertheless show that most entities are mapped correctly, while the lower macro-F1 indicates difficulty with small or underrepresented modules. W2V-FFM often appears stronger in the figure, particularly for C and C++ systems, but we do not interpret this as evidence that W2V initialization is generally superior to LPE. In this configuration, the initial W2V representation of a file is primarily derived from its filename, making it a lightweight identifier rather than a rich semantic description. The strong performance, therefore, also reflects how the full FFM graph shapes file embeddings through folder and member relations. In contrast, LPE directly encodes graph position from dependencies, which can also reflect noise when dependencies cross module boundaries. Therefore, the comparisons in this section mainly assess the effectiveness of the full FFM graph, while initialization choices are analyzed separately in RQ2.

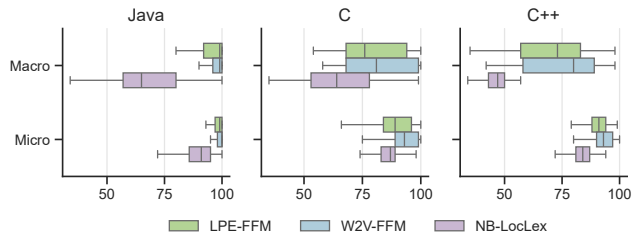


Figure 2: Distribution of macro-F1 and micro-F1 scores across repeated runs for the main full-graph *HetMap* variants and the NB-LocLex baseline, grouped by system language.

Answer to RQ1: *HetMap* is effective for semi-supervised code-to-architecture mapping under limited supervision. The full folder–file–member graph generally improves over the Naive Bayes baselines, indicating that HGT can propagate limited architectural supervision through typed folder, member, and dependency relations. The strongest results are obtained when the full heterogeneous graph is combined with informative node initialization.

5.2 Implementation Signals and Graph Representation Choices (RQ2)

To answer RQ2, we organize the experiments as controlled ablations over the heterogeneous graph representation. The goal is to understand which implementation signals provide architectural evidence, and whether their usefulness comes from the signal itself, from how it is represented in the graph, or from the node initialization strategy. We therefore vary one major design factor at a time: removing folder nodes to test the contribution of non-folder signals, randomizing folder tokens while preserving folder hierarchy to separate folder-name semantics from structural colocation, comparing file-only and file–member graphs to study member-level information, and comparing Word2Vec and LPE initialization to assess lexical and structural node features.

5.2.1 Folder-Free Graph Variants. We first examine a folder-free setting in which folder nodes and folder relations are removed from the graph. This ablation tests how much architectural evidence remains in file-level dependencies, member-level information, and node initialization features when folder organization is unavailable. NBA_{CDA} is included only as a baseline reference.

The results show a clear drop compared with the full FFM graph reported in Table 5, confirming that folder organization is an important architectural signal. However, several folder-free variants still outperform NBA_{CDA}, especially when LPE is used and when the datasets provide enough dependency structure. This indicates that static dependencies, member-level information, and structural positional encodings also contain useful architectural evidence, although they are less reliable than folder organization.

5.2.2 Folder Semantics versus Folder Structure. Next, we examine whether the benefit of folder information comes from meaningful folder names or from the folder hierarchy itself. For this purpose, we randomize folder tokens while preserving the original folder hierarchy and folder–file connections. In this setting, the model can still exploit file colocation and hierarchical structure, but it cannot use meaningful folder names such as `gui`, `model`, or `util`.

The comparison shows that folder tokens and folder hierarchy should be interpreted separately. When randomized folder-token variants remain competitive, the model is benefiting from hierarchical structure and file colocation rather than folder-name semantics alone. When semantic folder-token variants improve over randomized variants, the folder names themselves provide additional architectural evidence.

Table 6: Median macro-F1 (MA) and micro-F1 (MI) scores (%) for the folder-free ablation under the 5% low-label setting. NBA_{CDA} is included as a baseline reference. The remaining rows report *HetMap* variants without folder nodes or folder relations: file-only (F) and file–member (FM) graphs initialized with Word2Vec, LPE, and their combination.

Method	Java														C				C++			
	Ant		A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
NBA_{CDA}	16	44	31	62	20	53	40	81	53	75	48	73	23	60	21	44	45	55	–	–	45	72
$W2V_{F}$	20	40	30	60	17	41	43	82	57	74	33	53	45	72	14	33	36	38	14	55	27	46
$UnixCoder_{F}$	38	55	38	65	44	65	43	83	57	74	33	53	45	72	14	33	36	38	14	55	27	46
$W2V_{FM}$	19	45	32	61	17	50	41	81	57	75	19	40	28	61	21	37	44	40	16	57	35	58

Table 7: Median macro-F1 (MA) and micro-F1 (MI) scores (%) for folder-based graph variants with semantic and randomized folder tokens. Randomized variants preserve the folder hierarchy and folder–file connections while replacing meaningful folder-name tokens. The semantic FFM rows are reported in Table 5 as the main full-graph *HetMap* configurations.

Method	Java														C				C++			
	Ant		A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
$W2V_{FFM\text{-}Rand}$	85	93	94	98	68	83	85	97	89	93	100	100	100	100	68	88	93	93	87	97	54	89
$LPE_{FFM\text{-}Rand}$	85	93	94	98	66	80	81	97	87	92	100	100	100	100	69	84	88	90	84	94	55	88
$W2V_{FF}$	57	87	70	88	73	82	75	96	84	97	81	90	70	90	64	80	84	90	46	81	51	84
LPE_{FF}	70	88	74	89	83	85	78	96	85	98	80	91	75	91	70	86	81	89	70	89	51	83
$LPE+W2V_{FF}$	60	87	69	88	79	84	74	95	84	97	77	89	69	89	69	83	84	90	51	83	52	87
$W2V_{FF\text{-}Rand}$	57	87	64	86	72	82	67	95	84	97	84	92	70	90	63	79	84	90	48	83	52	85
$LPE_{FF\text{-}Rand}$	58	81	69	88	53	68	77	96	74	89	81	91	77	92	70	84	76	84	75	92	51	85

5.2.3 Lexical and Structural Node Initialization. Across the folder-free and folder-based variants, LPE is generally a stronger structural initializer than Word2Vec when folder information is reduced or unavailable. This is expected because LPE encodes positional patterns in the dependency graph, which are directly related to architectural structure. Word2Vec captures lexical information from files and member identifiers, but locally trained averaged token embeddings can make files with similar programming vocabulary appear close even when they belong to different modules. The combined LPE+W2V setting does not consistently dominate, suggesting that the two signals may be partly redundant or that lexical features can introduce noise in some systems.

5.2.4 Flattened Location and Member-Granularity Ablations. As a control, we also evaluate a location-informed setting in which folder/location tokens are added directly to file features. This differs from the main *HetMap* configuration, where folder information is represented through folder nodes and containment relations. The purpose of this experiment is to examine whether graph expressiveness still contributes when file nodes already have direct access to location information.

Answer to RQ2: Folder organization is the strongest architectural signal in *HetMap*. Removing folder nodes and folder relations substantially reduces performance, but folder-free variants still outperform NBA_{CDA} in several systems, showing that dependency structure, member-level information, and structural positional encodings also provide useful evidence. Semantic folder tokens usually improve performance, but randomized folder-token variants often remain competitive, indicating that hierarchy and file colocation are also important. Across variants, LPE is generally a stronger structural initializer than Word2Vec when folder information is limited, whereas Word2Vec is most useful when lexical information is represented alongside the full heterogeneous graph.

5.3 Scalability of *HetMap* (RQ3)

Answer to RQ3:

6 Related Work

Code-to-architecture mapping is a central prerequisite for architecture conformance checking and Reflexion modeling, in which

Table 8: Median macro-F1 (MA) and micro-F1 (MI) scores (%) for the flattened location-feature graph-granularity ablation. In all variants, folder/location tokens are added directly to file features. The rows compare increasing graph expressiveness: file-only (F), folder–file (FF), and folder–file–member (FFM) graphs. This setting tests whether explicit folder and member nodes still improve mapping performance when location information is already available at the file-feature level.

Method	Java														C				C++			
	Ant		A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
$W2V_{F+Loc}$	76	85	78	89	84	90	73	95	89	97	94	95	92	96	64	83	86	89	77	91	54	88
LPE_{F+Loc}	89	91	89	93	93	94	93	97	99	100	98	97	96	98	68	85	77	82	86	94	53	84
$W2V_{FF+Loc}$	86	93	88	96	93	93	93	97	99	100	98	97	99	100	67	83	94	94	81	92	55	88
$LPE_{FF+Loc+Loc}$	94	94	92	97	98	96	95	98	99	100	100	100	100	100	73	87	90	93	86	94	54	88
$W2V_{FFM+Loc}$	99	97	93	98	99	97	94	97	99	100	100	100	100	100	68	89	95	95	83	93	56	91
$LPE_{FFM+Loc}$	99	98	93	98	98	97	83	97	100	100	98	97	100	100	68	84	95	95	87	95	56	89

implementation-level dependencies are compared with architecture-level dependencies. Early work by Tzerpos and Holt [33] introduced the orphan adoption problem, in which unmapped source code entities are incrementally assigned to architectural modules. Building on this idea, Christl et al. Christl et al. [7, 8] proposed the Human-Guided Clustering Method (HuGMe), a semi-automated approach that uses attraction functions to recommend mappings between source-code entities and architectural modules.

Subsequent work explored different information sources and learning strategies for computing these mappings. Bittencourt et al. Bittencourt et al. [4] investigated information-retrieval-based attraction functions that rely on textual similarity between source-code entities and architectural modules. Bibi et al. Bibi et al. [3] evaluated supervised classifiers, including Naive Bayes, k-nearest neighbors, and neural networks, for the orphan adoption problem. Later, Olsson et al. [24, 26] introduced a Naive Bayes-based mapping approach that combines lexical information from source code with dependency-derived information through Concrete Dependency Abstraction (CDA). Their results showed that machine learning can improve over previous attraction-function-based approaches, but these methods still rely on engineered textual or dependency-derived representations rather than learned representations over typed software graphs.

Several studies have also investigated interactive, hierarchical, and text-based mapping. Florean et al. [11] compared machine learning-based text classifiers for assigning source code entities to architectural modules, showing that classifier choice and text representation influence mapping quality. Sinkala and Herold [30] proposed InMap, an interactive information-retrieval-based approach that recommends code-to-architecture mappings using natural-language architecture descriptions and source code, reducing the need for pre-mapped entities. This line of work was further extended by Sinkala and Herold [31], who integrated package- and class-level mapping using InMap, highlighting the importance of mapping granularity.

More recently, Johansson et al. [17] framed method-level code-to-architecture mapping as a text classification task and evaluated TF-IDF, GloVe, and LLM-based representations with classifiers such as Naive Bayes, support vector machines, and neural networks. In related LLM-based settings, Fuchß et al. [12] extracts architecture component names from documentation and source code to support traceability between architecture-level concepts and implementation entities, while Rukmono et al. [28] uses chain-of-thought prompting with GPT-4 to map implementation units to a predefined reference architecture. These studies show the value of LLMs and semantic representations for connecting implementation artifacts to architectural concepts. However, they mainly rely on textual representations, prompting, or intermediate architecture concepts, while the typed relational structure of software systems remains implicit. Prompting-based approaches may also face scalability challenges when applied to large codebases due to context-size limits and the cost of repeatedly processing implementation artifacts.

Graph-based representations have long been used in architecture recovery and software modularization, but they have only recently been studied for code-to-architecture mapping. In our previous work, we formulated source-code-to-architecture mapping as a semi-supervised node classification problem and evaluated GCN-based models on homogeneous and dependency-type-aware graphs [16]. That work demonstrated the potential of GNNs for entity mapping, but it did not model folder hierarchies, containment, or member-level artifacts within a heterogeneous software graph.

In contrast, *HetMap* models code-to-architecture mapping as semi-supervised learning over a heterogeneous software graph with typed artifacts and relations. This enables us to study the use of heterogeneous graph transformers for mapping and to analyze how different software signals, such as folder hierarchy, containment, dependencies, lexical features, and structural position, affect module assignment. To the best of our knowledge, these aspects have not been systematically studied for code-to-architecture mapping.

7 Threats to Validity

8 Conclusion

This paper presented *HetMap*, a heterogeneous graph learning framework for semi-supervised mapping of source code entities to architectural modules. By representing files, members, folders, dependency relations, containment, and hierarchy as typed graph elements, *HetMap* preserves architectural signals that are lost in

flattened representations. The experimental results show how heterogeneous relation modeling and feature initialization strategies, such as Word2Vec and LPE, affect mapping performance. Overall, the study supports heterogeneous graph learning as a promising approach to reducing manual effort in architecture conformance workflows.

References

[1] Nour Ali, Sean Baker, Ross O'Crowley, Sebastian Herold, and Jim Buckley. 2018. Architecture consistency: State of the practice, challenges and requirements. *Empirical Software Engineering* 23 (2018), 224–258.

[2] Len Bass, Paul Clements, and Rick Kazman. 2012. *Software Architecture in Practice*. Addison-Wesley.

[3] Maryum Bibi, Onaiza Maqbool, and Jaweria Kanwal. 2016. Supervised learning for orphan adoption problem in software architecture recovery. *Malaysian Journal of Computer Science* 29, 4 (2016), 287–313.

[4] Roberto Almeida Bittencourt, Gustavo Jansen de Souza Santos, Dalton Dario Serey Guerrero, and Gail C. Murphy. 2010. Improving Automated Mapping in Reflexion Models Using Information Retrieval Techniques. In *17th Working Conference on Reverse Engineering*. 163–172.

[5] Joao Brunet, Roberto Almeida Bittencourt, Dalton Serey, and Jorge Figueiredo. 2012. On the evolutionary nature of architectural violations. In *2012 19th Working conference on reverse engineering*. IEEE, 257–266.

[6] Jiayi Chen, Zhixing Wang, Yuchen Jiang, Jun Pang, Tian Zhang, Minxue Pan, and Jianwen Sun. 2022. Negar: network embedding guided architecture recovery for software systems. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, IEEE, 367–376.

[7] Andreas Christl, Rainer Koschke, and M-A Storey. 2005. Equipping the reflexion method with automated clustering. In *12th Working Conference on Reverse Engineering (WCRE'05)*. IEEE, 10–pp.

[8] Andreas Christl, Rainer Koschke, and Margaret-Anne Storey. 2007. Automated clustering to support the reflexion method. *Information and Software Technology* 49, 3 (2007), 255–274.

[9] Lakshitha De Silva and Dharini Balasubramaniam. 2012. Controlling software architecture erosion: A survey. *Journal of Systems and Software* 85, 1 (2012), 132–151.

[10] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2023. Benchmarking graph neural networks. *Journal of Machine Learning Research* 24, 43 (2023), 1–48.

[11] Alexander Florean, Laoa Jalal, Zipani Tom Sinkala, and Sebastian Herold. 2021. A Comparison of Machine Learning-Based Text Classifiers for Mapping Source Code to Architectural Modules. In *15th European Conference on Software Architecture*.

[12] Dominik Fuchß, Haoyu Liu, Tobias Hey, Jan Keim, and Anne Koziolek. 2025. Enabling architecture traceability by llm-based architecture component name extraction. In *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 1–12.

[13] Joshua Garcia, Ivo Krka, Chris Mattmann, and Nenad Medvidovic. 2013. Obtaining ground-truth software architectures. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 901–910.

[14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[15] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *Proceedings of the web conference 2020*. 2704–2710.

[16] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2025. Graph Convolution Networks for Mapping Source Code Entities to Architectural Modules. In *22nd IEEE International Conference on Software Architecture (ICSA 2025), Odense, Denmark, March 31-April 4, 2025*. IEEE.

[17] Nils Johansson, Mauro Caporuscio, and Tobias Olsson. 2024. Mapping Source Code to Software Architecture by Leveraging Large Language Models. In *European Conference on Software Architecture*. Springer, 133–149.

[18] Jens Knodel and Daniel Popescu. 2007. A comparison of static architecture compliance checking approaches. In *Working IEEE/IFIP conference on software architecture*. 12–12.

[19] Jörg Lenhard, Martin Blom, and Sebastian Herold. 2019. Exploring the suitability of source code metrics for indicating architectural inconsistencies. *Software Quality Journal* 27, 1 (2019), 241–274.

[20] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidović, and Robert Kroeger. 2017. Measuring the impact of code dependencies on software architecture recovery techniques. *IEEE Transactions on Software Engineering* 44, 2 (2017), 159–181.

[21] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26 (2013).

[22] Gail C Murphy, David Notkin, and Kevin Sullivan. 1995. Software reflexion models: Bridging the gap between source and high-level models. In *3rd ACM SIGSOFT symposium on Foundations of Software Engineering*. 18–28.

[23] Hung Viet Nguyen. 2023. Depends: Multilingual Dependency Analysis Tool. https://github.com/multilang-depends/depends. Accessed: 2025-08-17.

[24] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2019. Semi-automatic mapping of source code using Naive Bayes. In *Proceedings of the 13th European Conference on Software Architecture-Volume 2*. 209–216.

[25] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2021. s4rdm3x: A Tool Suite to Explore Code to ArchitectureMapping Techniques. *Journal of Open Source Software* 6, 58 (2021).

[26] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2022. To automatically map source code entities to architectural modules with Naive Bayes. *Journal of Systems and Software* 183 (2022), 111095.

[27] Mark Richards and Neal Ford. 2025. *Fundamentals of Software Architecture: A Modern Engineering Approach*. " O'Reilly Media, Inc.".

[28] Satrio Adi Rukmono, Lina Ochoa, and Michel Chaudron. 2024. Deductive software architecture recovery via chain-of-thought prompting. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 92–96.

[29] Anthony Savidis and Crystalia Savaki. 2022. Software Architecture Mining from Source Code with Dependency Graph Clustering and Visualization.. In *VISIGRAPP (3: IVAPP)*. 179–186.

[30] Zipani Tom Sinkala and Sebastian Herold. 2021. InMap: Automated Interactive Code-to-Architecture Mapping Recommendations. In *18th International Conference on Software Architecture*. 173–183.

[31] Zipani Tom Sinkala and Sebastian Herold. 2023. An integrated approach to package and class code-to-architecture mapping using InMap. In *2023 IEEE 20th International Conference on Software Architecture (ICSA)*. IEEE, 164–174.

[32] Ioanna Stavropoulou, Marios Grigoriou, and Kostas Kontogiannis. 2017. Case study on which relations to use for clustering-based software architecture recovery. *Empirical Software Engineering* 22, 4 (2017), 1717–1762.

[33] Vassilios Tzerpos and Richard C Holt. 1997. The orphan adoption problem in architecture maintenance. In *Working Conference on Reverse Engineering*. IEEE, 76–82.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[35] Yiran Zhang, Zhengzi Xu, Chengwei Liu, Hongxu Chen, Jianwen Sun, Dong Qiu, and Yang Liu. 2023. Software architecture recovery with information fusion. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1535–1547.